Maven Rajesh D

Introduction

In the Java ecosystem there are 2 build tools which are widely used.
As a developer, I do the following things.

- Create new projects
- Manage dependencies and their versions
    - Spring, Spring MVC, Hibernate
    - Add/Modify dependencies
- **Build** a JAR file
- Run your application locally in Tomcat or jetty or any other

- Run **unit tests**
- Deploy to a test environment
- **…. so on!!**

Following both build tools help us to do all above.

Maven

- It can manage project's build , reporting and documentation from a central piece of information.

File Structure

- Java files
    - **src/main/java**
- Application resources
    - **src/main/resources**
- Test files
    - **src/test/java**

Step 1 Creating spring boot project with maven

Step 2 pom.xml (Project Object Mode)

pom.xml contains,

- dependencies of our project
- parent pom
- name of our project

Maven dependencies

- Frameworks and libraries used in our project
- Defined under **<dependencies></dependencies>**
- Each dependency can have a number of other dependencies which are called **transitive dependencies.**
- Once you add a dependency in pom.xml, maven downloads all its transitive dependencies and it makes sure that jars are available for application.

Parent pom of Spring boot project

- the groupId and artifact id that we put in parent tag are for other projects which want to use our project as dependency.
- **Name of our project** : groupId + artifactId
    - GroupId - similar to package name
    - artifactId - similar to class name
- Once we build our project and push it to maven repository, then other projects can make use of groupId, artifactId to add our project as dependency
- Whenever you face a problem with pom.xml, then try to lookout for **effective pom**

Basic Structure of <dependency> Tag

```
<dependency>
   <groupId>GROUP_ID</groupId>
   <artifactId>ARTIFACT_ID</artifactId>
   <version>VERSION</version>
   <scope>SCOPE</scope>  <!-- (Optional) Defines usage scope -->
   <optional>true/false</optional>  <!-- (Optional) Marks as optional -->
</dependency>
```

Required Elements

<groupId>

Unique identifier for the library's organization (e.g., org.springframework.boot).

<artifactId>

Specific module or project within the group (e.g., spring-boot-starter-web).

<version>

The version of the library (e.g., 3.4.3).

Optional Elements

<scope>

Defines where the dependency will be used (e.g., compile, test, provided).

<optional>

Marks the dependency as optional (used in multi-module projects)

Dependency Scope in Maven

Scope	Usage	Example
compile (default)	Used in all phases (compilation, testing, runtime)	General dependencies like `spring-core`
provided	Required for compilation but available at runtime (e.g., servlet API)	`javax.servlet-api`
runtime	Needed only at runtime, not for compilation	JDBC drivers (`mysql-connector-java`)
test	Used only for testing purposes	JUnit (`junit-jupiter-api`)
system	Similar to provided, but requires a local JAR path	Rarely used

Step 6 Maven Build Life Cycle

- Whenever we run a maven command, the maven build life cycle is used.
- Build LifeCycle is a sequence of steps,
    - **Validate** - is everything good ?
    - **Compile** - compile the java code. Compiled classes are available in /target folder
    - **Test** - run your unit tests
    - **Package** - create a jar file
    - **Integrations Test** - run you integrations tests
    - **Verify**
    - **Install**
    - **Deploy**

Step 7 How does maven work ?

- Maven follows Convention over Configuration
    - Predefined folder structure
    - Almost all java projects follow **Maven structure** (Consistency)

Maven central repository

- It contains all jars (and others) indexed by artifactId and groupId
- We can add our own repository urls (which usually companies follows)
- By default mavens looks in the central repository. If it doesn't find it, then it checks in our own repositories if we add any.
- When a dependency is added to pom.xml, maven tries to download the dependency
    - Downloaded dependencies are stored inside your maven **local repository**

Local Repository

- A temp folder on your machine where maven stores the jar and dependencies files that are downloaded from Maven repository

Step 8 Maven Commands

- **mvn --version**
- **mvn compile** :- Compiles source files
- **mvn test-compile** :- Compiles test files
    - Test compile also compiles source files.
- **mvn clean** :- Deletes target directory
- **mvn test** :- Run unit tests
- **mvn package** :- Creates a jar file
- **mvn help:effective-pom** :- generates effective pom
- **mvn dependency:tree** :- prints dependency tree

Step 9 How Spring Releases are versioned ?

Version-scheme:-

- MAJOR.MINOR.PATCH[-MODIFIER] //modifier is optional
- **Major** : Significant amount of work to upgrade (10.0.0 to 11.0.0)
- **Minor** : Little to no work to upgrade (10.1.0 to 10.2.0)
- **Patch** : No work to upgrade (10.5.4 to 10.5.5)
- **Modifier** : Optional modifier
    - **Milestones** - M1, M2… (10.3.0 - M1, 10.3.0-M2)
    - **Release candidates** - RC1, RC2 (10.3.0-RC1, 10.3.0-RC2)
    - **Snapshots** - SNAPSHOT
        - Note: You should never use snapshots in your project
    - **Release** - Modifier will be ABSENT (10.0.0, 10.1.0)
- Example versions in order:

    - 10.0.0-SNAPSHOT **-->** 10.0.0-M1 **-->** 10.0.0-M2 **-->** 10.0.0-RC1 **-->** 10.0.0-RC2 **-->** 10.0.0
- Recommendations 🚨
    - Avoid snapshots ❌
    - Use only Released versions in PRODUCTION 😈